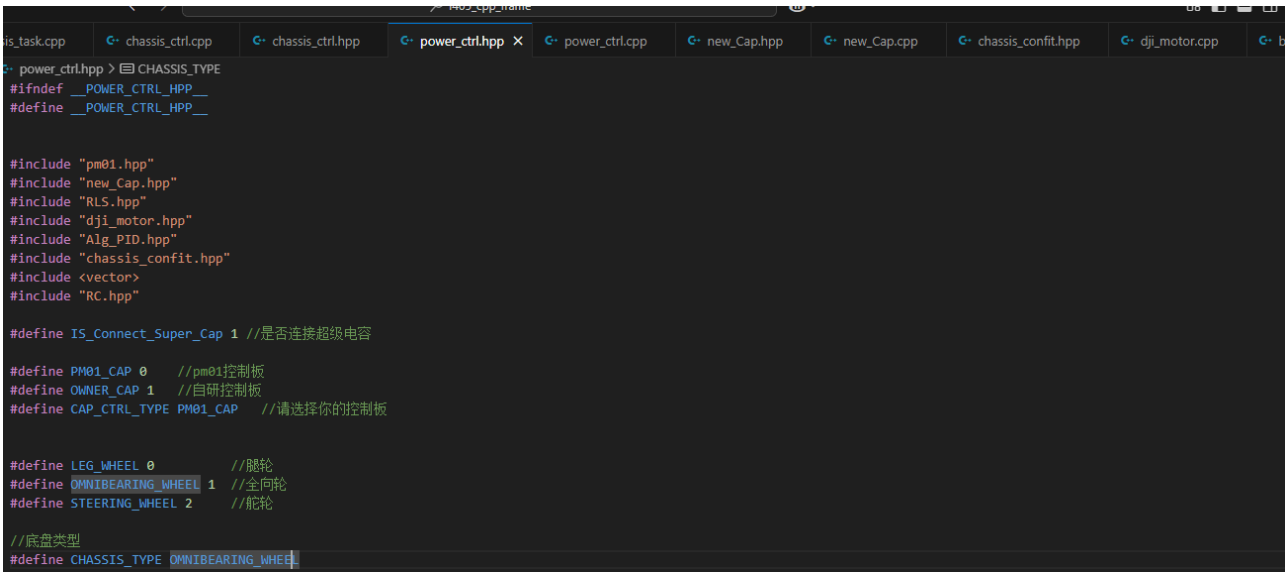
功率控制层使用

首先选择是否连接超电、功率控制板以及你的底盘类型



初始化，传入底盘四个电机的指针

```
std::vector<const DJI_Motor_n::DJI_Motor_Instance *> motors_;
    motors_.push_back(chassis_ctrl_c::Get_Chassis_Instance()-
>wheel_motor[0]);
    motors_.push_back(chassis_ctrl_c::Get_Chassis_Instance()-
>wheel_motor[1]);
    motors_.push_back(chassis_ctrl_c::Get_Chassis_Instance()-
>wheel_motor[2]);
    motors_.push_back(chassis_ctrl_c::Get_Chassis_Instance()-
>wheel_motor[3]);

    Core::Control::Power::Manager manager(motors_,
Core::Control::Power::Division::INFANTRY);// 功率管理器实例
    Core::Control::Power::init(manager);
```

以下函数放在循环任务中，用于更新RLS参数等

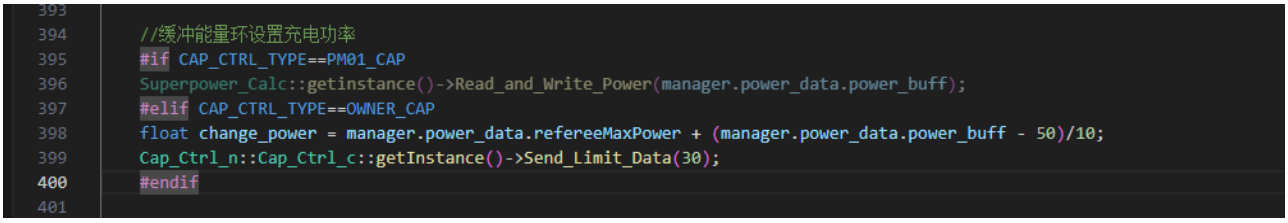
```
Core::Control::Power::Power_Ctrl_Loop();
```

功率控制过程

缓冲能量环

缓冲能量环用于控制从电管拿过来的功率，在保证不会耗尽缓冲能量的前提下合理利用剩余缓冲能量

这部分可以研究一下更好的方案。



```
393
394     //缓冲能量环设置充电功率
395     #if CAP_CTRL_TYPE==PM01_CAP
396     Superpower_Calc::getinstance()->Read_and_Write_Power(manager.power_data.power_buff);
397     #elif CAP_CTRL_TYPE==OWNER_CAP
398     float change_power = manager.power_data.refereeMaxPower + (manager.power_data.power_buff - 50)/10;
399     Cap_Ctrl_n::Cap_Ctrl_c::getInstance()->Send_Limit_Data(30);
400     #endif
401
```

功率环（需要用户调用）

调用getControlledOutput函数，传入每个轮的当前真实转速、设置转速以及pid计算的输出，会返回功率重新分配后的新设定值。

使用案例：

```
    static Core::Control::Power::PowerObj objs[4];
    static std::vector<Core::Control::Power::PowerObj *>pObjs =
{&objs[0], &objs[1], &objs[2], &objs[3]};
    // 更新功率对象
    for (int i = 0; i < 4; i++)
    {
        objs[i].curAv     = rpm2av(wheel_motor[i]-
>MotorMeasure.measure.feedback_speed/M3508_RATIO);  //电机实际转速
        objs[i].setAv     =
rpm2av(wheel_data[i].motor_set_ref/M3508_RATIO);//电机设定转速
        objs[i].pidOutput = wheel_data[i].motor_give_current;  //要
发送给电机的PID计算结果
        objs[i].pidMaxOutput = 16300;    //最大输出
    }
    float *controlledOutput =
Core::Control::Power::getControlledOutput(pObjs); //获得新的设定值
    for (int i = 0; i < 4; i++)
    {
        wheel_data[i].motor_give_current = controlledOutput[i];
```

```
    }
```

能量环

根据manager.power_mode_决定

用户模式：可以通过void setMaxPowerConfigured(float maxPower)函数设置底盘输出功率上限

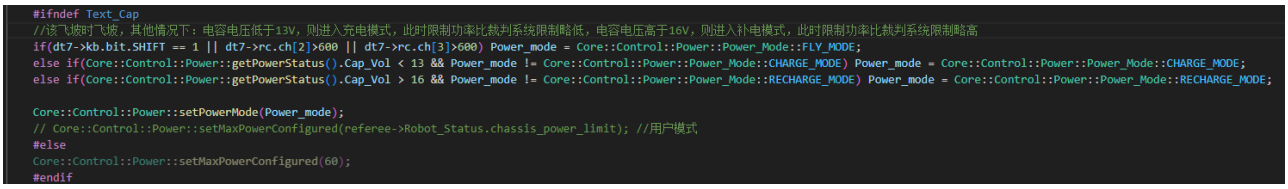
其他情况下，如果注册了回调函数，可以设置自己需要的功率曲线，否则：

充电模式：在底盘运动的时候缓慢补充超电能量

补电模式：底盘运动情况下缓慢消耗超电能量

飞坡模式（步兵用，其他兵种可以当成高功率模式）：功率上限设置大大增加

这是我目前使用的：



```
#ifndef Text_Cap
//该飞坡时飞坡，其他情况下：电容电压低于13V，则进入充电模式，此时限制功率比裁判系统限制略低，电容电压高于16V，则进入补电模式，此时限制功率比裁判系统限制略高
if(dt7->kb.bit.SHIFT == 1 || dt7->rc.ch[2]>600 || dt7->rc.ch[3]>600) Power_mode = Core::Control::Power::Power_Mode::FLY_MODE;
else if(Core::Control::Power::getPowerStatus().Cap_Vol < 13 && Power_mode != Core::Control::Power::Power_Mode::CHARGE_MODE) Power_mode = Core::Control::Power::Power_Mode::CHARGE_MODE;
else if(Core::Control::Power::getPowerStatus().Cap_Vol > 16 && Power_mode != Core::Control::Power::Power_Mode::RECHARGE_MODE) Power_mode = Core::Control::Power::Power_Mode::RECHARGE_MODE;

Core::Control::Power::setPowerMode(Power_mode);
// Core::Control::Power::setMaxPowerConfigured(referee->Robot_Status.chassis_power_limit); //用户模式
#else
Core::Control::Power::setMaxPowerConfigured(60);
#endif
```

这套功率控制的上限比较高，缓冲能量环和能量环都可以优化，期待探索